DATABASE SYSTEMS

Database Concurrency

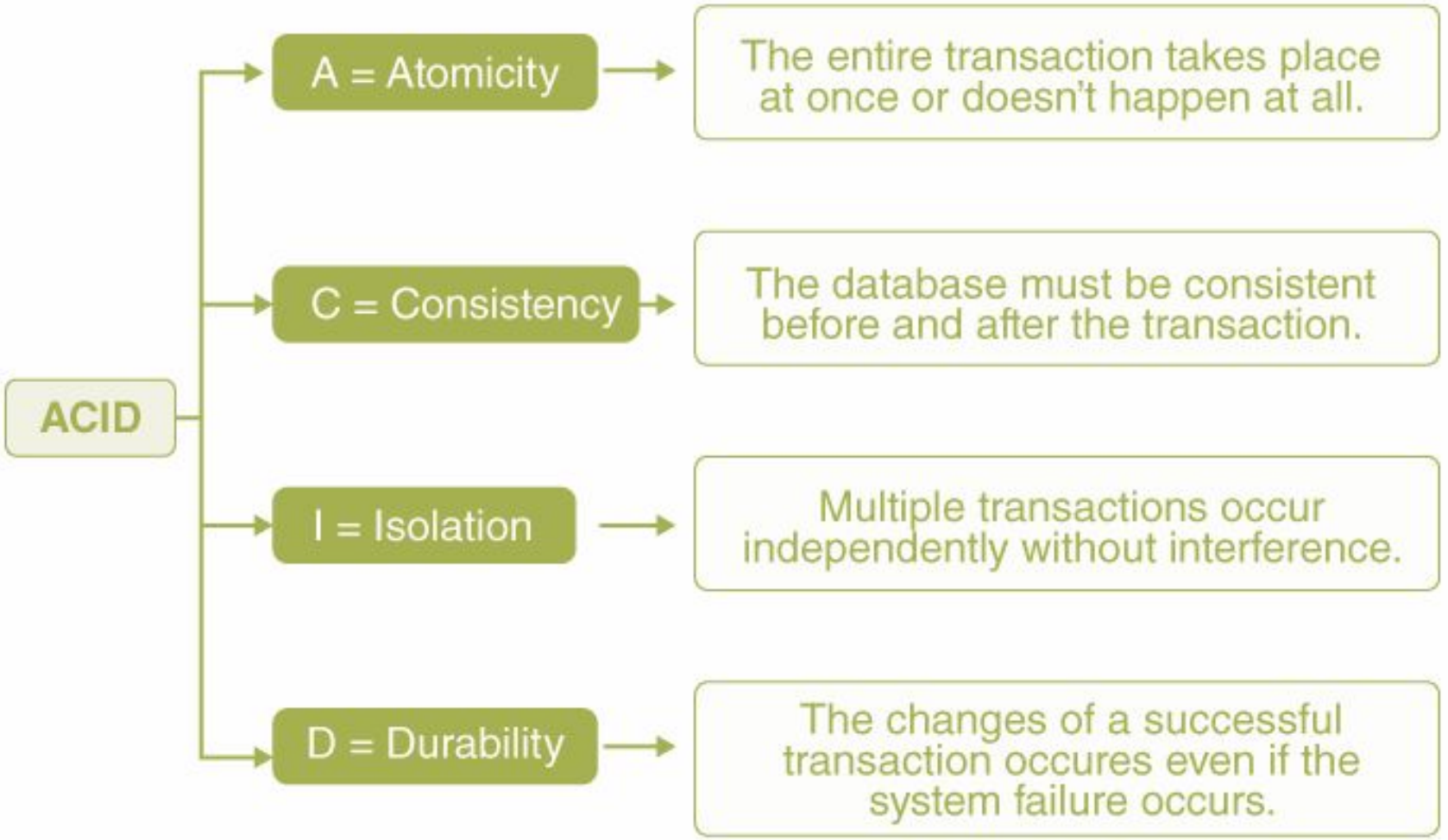
Faculty of AI & MMG

Concurrency Control in Databases

- The **transaction** refers to a small unit of any given program that consists of various low-level tasks.
- Every transaction in DBMS must maintain ACID – A (Atomicity), C (Consistency), I (Isolation), D (Durability).
- One must maintain ACID so as to ensure completeness, accuracy, and integrity of data.

- **Concurrency control** ensures that multiple transactions can execute simultaneously without leading to data inconsistency.
- It maintains the correctness of data when multiple users access/modify it concurrently.

ACID Properties in DBMS



Atomicity

Bank transfer: if adding to Account B fails, the deduction from Account A must also be undone.

- Atomicity means a transaction is completed fully or not executed at all.
- It follows the all-or-nothing rule.
- If any step fails, completed steps are undone through rollback.

Example
Bank transfer:
Deduct money from Account A.
Add money to Account B.
If step 2 fails, step 1 must be undone.

Consistency

Example rule: balance must not be negative.

- Consistency means every transaction preserves database rules and constraints.
- A transaction must take the database from one valid state to another valid state.
- Constraints, triggers, and application rules help enforce consistency.

Example

A bank account balance should never become negative if the rule says minimum balance must be Rs. 0.

SQL

```sql
CHECK (balance >= 0)
```

If a transaction violates this rule, the DBMS should reject it.

Isolation

Two users buying the last ticket should not both succeed.

- Isolation means transactions should not interfere with each other.
- Each transaction should behave as if it is running alone.
- Isolation is the ACID property most directly supported by concurrency control.

Example
Two users buying the last ticket should not both succeed.
The DBMS controls access so only one transaction can complete the booking.
Isolation is the ACID property most closely related to concurrency control.

Durability

After payment confirmation, the payment record should survive a power failure.

- Durability means committed changes are permanent.
- Once a transaction commits, the DBMS must preserve the result even after a crash.
- Logs, checkpoints, backups, and recovery mechanisms support durability.

Example
After payment is completed and confirmed, the database must remember it even if:
Power fails
Server restarts
System crashes
Durability is usually achieved using logs, backups, and recovery systems.

Problems Without Concurrency Control

Problem	What happens	Simple example
Lost update	One write overwrites another write	Two users update the same balance from an old value
Dirty read	A transaction reads uncommitted data	T2 reads a value that T1 later rolls back
Non-repeatable read	Same row gives different values inside one transaction	T1 reads balance twice; T2 updates between reads
Phantom read	A repeated query returns new rows	T2 inserts a matching row between T1 queries
Incorrect summary	Aggregate uses mixed old and new data	Report sums balances while transfers are running

Key Concepts in Concurrency Control:

- Locking: This involves using locks to control access to data items, ensuring that only one transaction can modify a specific resource at a time.
- Timestamp Ordering: This method assigns a timestamp to each transaction and orders them based on these timestamps to resolve conflicts.
- Deadlock Prevention: This involves detecting and resolving deadlocks, which can occur when two or more transactions are blocked indefinitely, waiting for each other to release resources.

Deadlocks & Serializability

- **Deadlocks**: Occur when transactions wait indefinitely for each other's locks.
- *Example*: T1 holds A and waits for B; T2 holds B and waits for A.
  - **Solution**: Timeouts or deadlock detection (aborting one transaction).

- **Serializability:** Ensures concurrent transactions produce the same result as some serial execution.
  - Conflict Serializability: Reorder transactions using swap of non-conflicting operations.
  - View Serializability: Less strict, allows blind writes.